

06 — Capability Planner

Objetivo. Documentar cómo Puglit *decide qué construir*: el camino que va de una idea de una línea ("un Airbnb para coworkings", "un ERP hospitalario") hasta el conjunto exacto de tablas, rutas, páginas y **módulos** que terminan ensamblados en la app generada — sin que el usuario enumere ninguna feature. El planner es el órgano que traduce *intención* en *capabilities* verificables, y después garantiza que ese conjunto esté completo (sin features faltantes) y acotado (sin sobre-ingeniería).

Todo lo que sigue está verificado contra el código en `/Users/alvaroz/projects/2026/puglit: web/lib/app-builder.ts` (`planBlueprint`, `studyReference`, las inyecciones `deterministicX` en `finalize`), `web/lib/swarm-planner.ts` (`planCapabilities`, `resolveDeps`), y `web/lib/module-registry.ts` (`BUILTIN_MODULES`, `MODULE_REQUIRES`, `dependencyClosure`).

1. Intent Analysis

El input de Puglit es deliberadamente pobre: una idea de una línea. Toda la riqueza —tablas, endpoints, pantallas, integraciones— hay que *inferirla*. El primer paso es entender la intención del producto antes de comprometerse a una arquitectura.

El **Domain Architect** (`planBlueprint`, `web/lib/app-builder.ts`) recibe un `DomainConfig` (nombre, tagline, lenguajes, entidades-hint, monetización) más los `contracts`, y se le pide *pensar en los user journeys reales* y devolver el blueprint funcional **completo**: las tablas, las operaciones de API y las páginas que un usuario real necesita para *usar el producto end-to-end* — no un CRUD admin genérico.

El system prompt no le pide al modelo que adivine en el vacío: le inyecta **ejemplos de inferencia literales** que anclan el razonamiento. Tres de ellos, tomados textualmente del código:

Idea de una línea	Inferencia esperada (resumida)
Tinder-style used-goods marketplace	tablas <code>items</code> , <code>swipes</code> , <code>matches(user_a,user_b,item_a,item_b)</code> , <code>messages(match_id,...)</code> ; operaciones: publicar item, feed de swipe (items de otros no swipeados), registrar swipe y detectar match mutuo , listar matches, chat por match. Anónimo: nunca exponer el email entre usuarios.
Booking / rental marketplace (Airbnb/hotel/coworking)	<code>listings</code> , <code>listing_photos</code> , <code>availability_blocks</code> , <code>bookings</code> , <code>payments</code> , <code>reviews</code> , <code>messages</code> . Plata en enteros de centavos . La lógica dura (overlap, anti-double-booking, pricing determinista, refund-by-policy-snapshot, reviews double-blind) ya está pre-construida en <code>lib/rentals/*</code> — el architect debe importar , no reimplementar.
Location + membership aggregator ("qué lugares cerca me dan descuento con mis tarjetas")	<code>programs</code> , <code>user_memberships</code> , <code>merchants</code> , <code>branches(lat,lng DOUBLE PRECISION)</code> , <code>offers(program_id)</code> . La ruta core: "ofertas cercanas para MIS programas" — Haversine en SQL ordenado por distancia. El catálogo es ingestado , no user-generated.

Estos ejemplos no son decoración: enseñan al modelo a **distinguir patrones de dominio** (un swipe-marketplace necesita match mutuo atómico; un agregador geo necesita lat/lng en `DOUBLE PRECISION` y Haversine). El prompt cierra con una **regla general** explícita: separar *CATALOG/reference data* (curada o ingestada de fuentes externas — sembrarla, refrescarla por cron, los usuarios solo la leen) de *USER-GENERATED content* (los usuarios la crean). Modelar ambas bien.

2. Domain Classification (public vs accounts, data-driven)

Antes de cualquier tabla, el architect toma **una** decisión que reordena toda la app. El `Blueprint` lleva un campo `kind: "public" | "accounts"` (`web/lib/app-builder.ts`, interfaz `Blueprint`):

- public** — el producto es la homepage, usable sin cuenta (un scoreboard deportivo, un feed de noticias, una herramienta pública, un directorio). **Sin** login, **sin** signup, **sin** pricing. La home en `/ (app/page.tsx)` renderiza el producto real; las rutas de API son **públicas** (sin auth).
- accounts** — el valor central es data privada por-usuario (un marketplace con *mis* listings, una app social, un dashboard personal). Tiene signup/login y data por-usuario detrás de auth.

Esta clasificación es **data-driven**: el prompt instruye "*elegí honestamente desde el concepto; la mayoría de los 'clon de <sitio público>' o herramientas son public*". La decisión no es cosmética — propaga por todo el pipeline:

- Backend** (`genRouteFile`): si `kind === "public"`, al agente de backend se le dice "*NO llames a `getAuthUser`; las rutas son lecturas/escrituras públicas sobre el catálogo*"; si `accounts`, "*protegé las rutas por-usuario con `getAuthUser` (401 si falta)*".

- **Frontend** (`genPage`): la home pública renderiza el producto real (sin hero de marketing, sin "Empezá gratis", sin pricing); en `accounts` se inyecta el **AUTH GATE** (si un fetch devuelve 401, `router.replace("/login")`).
- **Finalize**: solo los productos `accounts` reciben `ensureAuthPages` y un app-shell con navegación autenticada (`genAppShell / fallbackShell`).

La regla *data-driven* tiene un corolario importante: un producto que muestra **datos externos/live** (scores, precios, vuelos, listings) se modela como un **catálogo curado refrescado por cron**, nunca como user-generated, y se asume un job de ingestión que lo puebla. Esto evita el error clásico del generador: pedirle al usuario que cargue a mano los partidos de fútbol de un clon de Promiedos.

3. Capability Extraction (`planCapabilities`)

El blueprint dice *qué entidades y pantallas* tiene el producto. Pero Puglit tiene ~85 **módulos** pre-construidos (pagos, billing, RAG, agente, multitenancy, auditlog, wallet, 2FA...) que no hay que volver a generar. ¿Cómo se decide cuáles inyectar?

El mecanismo base es **keyword matching**: cada módulo tiene una función `deterministicX(config, bp)` que arma un *haystack* de texto y matchea un regex (ver §5). El problema, anotado como crítica explícita en el código: **la inyección por keyword es frágil**. Un "ERP para hospitales" no contiene ninguno de los keywords de `auditlog`, `multitenancy` o `entitlements`, aunque obviamente los necesita.

La respuesta es `planCapabilities` (`web/lib/swarm-planner.ts`):

```
export async function planCapabilities(productText: string, catalog: string): Promise<string[]>
```

Es un **LLM planner** (corre en `MODELS.premium`, `temperature: 0.1`) cuyo system prompt es:

"You are a capability planner for an app generator. Given a product description and a catalog of available modules, return ONLY the module names whose capability the product genuinely needs (think about implicit needs: an ERP needs auth+multitenancy+audit even if unsaid). Be precise, do not over-select."

Devuelve JSON `{ modules: string[] }` con schema forzado, normaliza a minúsculas/trim y filtra vacíos. Es **defensivo**: cualquier error devuelve `[]` (degrada con gracia, nunca rompe el build).

El detalle clave de **integración** está en `finalize` (`app-builder.ts`):

```
const planned = await planCapabilities(
  `${config.identity.name} ${tagline} ${bp.summary}`,
  await moduleCatalog()).catch(() => [])
if (planned.length) bp.summary += ` [capabilities: ${planned.join(" ")}`
```

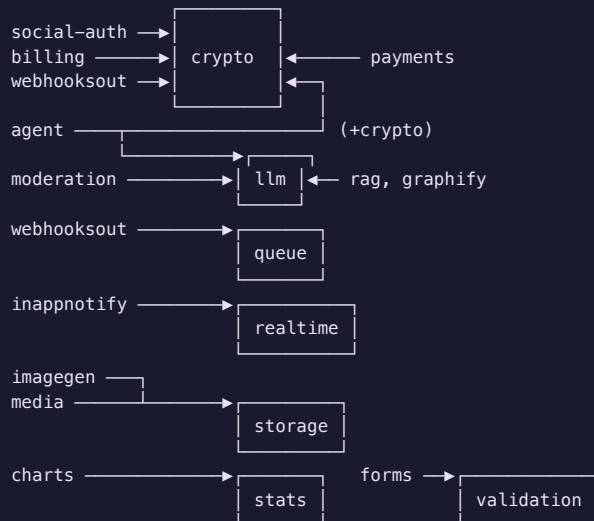
El planner **no inyecta módulos directamente**. Lo que hace es *augmentar el* `bp.summary` con una cola `[capabilities: auditlog multitenancy entitlements ...]`. Como los `deterministicX` arman su *haystack* incluyendo `bp.summary` (§5), los inyectores deterministas **disparan también para las capabilities planeadas** que el regex hubiera perdido. Es una arquitectura elegante: el LLM no reemplaza al mecanismo determinista, lo *alimenta* — el camino frágil (regex) y el camino robusto (LLM) convergen en el mismo punto de inyección.

4. Capability Graph

Las capabilities no son independientes: un módulo puede *requerir* otro para funcionar (si guardás tokens OAuth de social-auth, necesitás `crypto` para no dejarlos en plaintext). Puglit modela esto como un **grafo de dependencias duras** (`MODULE_REQUIRES`, `web/lib/module-registry.ts`):

```
export const MODULE_REQUIRES: Record<string, string[]> = {
  "social-auth": ["crypto"],   billing: ["crypto"],   payments: ["crypto"],
  inappnotify: ["realtime"],   moderation: ["llm"],   rag: ["llm"],
  agent: ["llm", "crypto"],    charts: ["stats"],    forms: ["validation"],
  webhooksout: ["crypto", "queue"], imagegen: ["storage"], media: ["storage"],
  graphify: ["llm"],
}
```

Diagrama del grafo (cada arista = "requiere"):



Las **hojas** del grafo (`crypto`, `llm`, `queue`, `realtime`, `storage`, `stats`, `validation`) son exactamente los siete módulos que el resolver sabe force-inyectar (`DEP_INJECTORS` en `swarm-planner.ts`). Son "infra" reutilizable: zero-dep, sin keywords propios, traídos *por necesidad de otro módulo*, no por match directo.

El cierre transitivo del grafo lo calcula `dependencyClosure`:

```

export function dependencyClosure(names: string[]): string[] {
  const seen = new Set(names); const stack = [...names]
  while (stack.length) { const n = stack.pop()!
    for (const dep of MODULE_REQUIRES[n] || []) if (!seen.has(dep)) { seen.add(dep); stack.push(dep) }
  }
  return [...seen]
}

```

Es un DFS sobre conjunto-visitado: idempotente, robusto a ciclos (el `Set` corta la recursión), devuelve el conjunto completo de módulos requeridos partiendo de los presentes.

5. Module Selection (keyword matching de los `deterministicX` + planner LLM)

La selección efectiva ocurre en `buildAdvance` durante la fase `finalize` (`app-builder.ts`): ~70 inyectores `deterministicX(config, bp)` se llaman en secuencia, cada uno decidiendo si dispara.

El patrón de cada inyector es uniforme. Ejemplo verificado de `deterministicPayments` (`web/lib/payments-module.ts`):

```

export function deterministicPayments(config, bp) {
  const hay = `${config.identity.name} ${tagline} ${bp.summary} ${bp.tables.map(t=>t.name).join(" ")} ${config.monetization} ||
  const wants = /pago|payment|checkout|stripe|mercadopago|suscrip|subscription|cobr|charge|precio|price|plan|premium|paywall|
  if (!wants) return null
  return { files: [...], extraSql: PAYMENTS_SQL }
}

```

Dos cosas a notar:

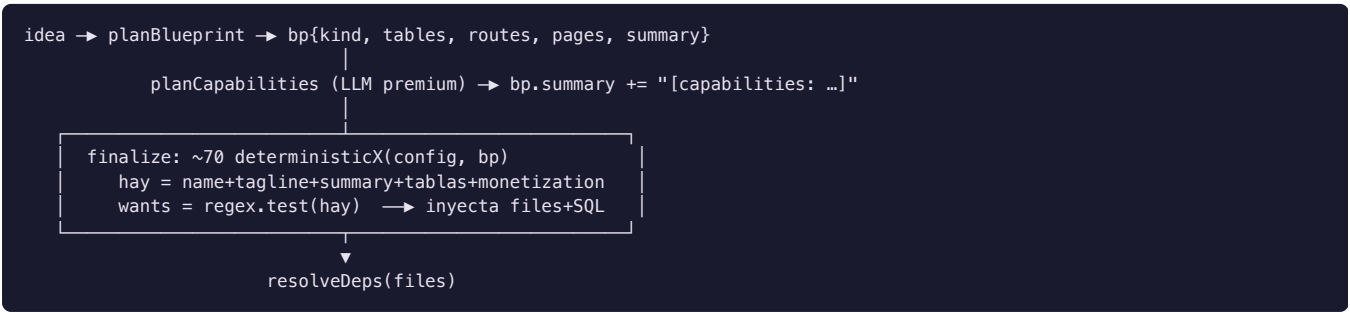
1. **El haystack incluye `bp.summary`**. Por eso `planCapabilities` (§3) puede activar un inyector sin tocar su regex: basta con que el nombre del módulo (p.ej. `payments`) aparezca en la cola `[capabilities: ...]` que el planner agregó al summary. Algunos inyectores también suman `bp.tables` (los nombres de tabla) y `config.monetization` al haystack, ampliando la superficie de match.
2. **El regex es bilingüe (es/en)** — `pago|payment`, `suscrip|subscription`, `cobr|charge` — porque los productos llegan en español rioplatense o inglés indistintamente.

Otros patrones verificados, para ilustrar la cobertura:

Módulo	Fragmento del regex <code>wants</code> (bilingüe)
booking	book booking reserv cita appointment schedul agenda turno slot calendar rental alquiler disponibilidad
agent	asistente assistant chief of staff jarvis copilot ai agent chatbot \bbot\b second brain
rag	rag semantic embedding vector knowledge docs ask your chat with q&a faq recomend recommend
auditlog	audit compliance cumplimiento soc2 gdpr hipaa trail fintech banking legal admin
wallet	credit cr[eé]dito wallet billetera points puntos token saldo loyalty prepaid recarga
twofa	2fa mfa two.?factor totp authenticator otp secure login banking fintech wallet
entitlements	plan tier entitle gating premium upgrade suscrip subscription saas paywall

Cada inyector que dispara aporta `files` (código del módulo) y, opcionalmente, `extraSql` que se appendea a `sql/app.sql` *idempotentemente* (un helper `addSql(marker, label, sql)` chequea un regex marker para no duplicar la tabla si ya está). Los archivos se pushean solo si su `path` no existe ya (`pushFiles`), así un módulo nunca pisa un archivo que el swarm ya escribió.

Resumen del flujo de selección:



6. Dependency Expansion (`resolveDeps` , `dependencyClosure`)

Después de que todos los inyectores corrieron, queda un riesgo: un keyword disparó `social-auth` pero nadie disparó `crypto`, y la app shipea guardando tokens en plaintext. Eso lo cierra `resolveDeps` (`web/lib/swarm-planner.ts`), llamado en `finalize` justo después de las inyecciones de módulos (antes de `voice/agent`):

```
export function resolveDeps(files: AppFile[]): string[] {
  const has = (p) => files.some(f => f.path === p)
  const present = []
  for (const name of Object.keys(MODULE_REQUIRES))
    if (has(PRIMARY[name] || `lib/${name}.ts`)) present.push(name) // 1. detectá qué módulos hay
  const needed = dependencyClosure(present) // 2. cierre transitivo
  const added = []
  for (const dep of needed) {
    const inj = DEP_INJECTORS[dep]
    if (!inj || has(depFile[dep] || `lib/${dep}.ts`)) continue // ya presente → skip
    const r = inj() // 3. force-inyectá la base
    for (const f of r.files) if (!has(f.path)) files.push(f)
    if (r.extraSql) { /* append idempotente a sql/app.sql */ }
    added.push(dep)
  }
  return added
}
```

El algoritmo es **detección por archivo primario**: en vez de confiar en un flag de estado, mira los archivos *realmente presentes*. El mapa `PRIMARY` dice cuál es el archivo-firma de cada módulo dependiente (`social-auth` → `lib/social/providers.ts`, `agent` → `lib/agent/brain.ts`, `rag` → `lib/rag.ts`...). Si ese archivo está, el módulo está. Después calcula el cierre transitivo con `dependencyClosure` (§4) y, para cada dependencia faltante, llama su inyector *ungated* (`DEP_INJECTORS`: `crypto`, `storage`, `realtime`, `llm`, `queue`, `stats`, `validation`) — sin pasar por ningún regex, porque la necesidad ya está probada por el grafo. Loggea las que agregó: `[deps] auto-injected: crypto, ...`.

Esta separación —*selección* por keyword/LLM, *expansión* por grafo determinista— es lo que garantiza que **un build nunca shipea con su módulo base faltante**.

7. Conflict Detection

Puglit prefiere **prevenir conflictos por construcción** antes que detectarlos a posteriori. Los mecanismos verificados:

- **Idempotencia de archivos.** `pushFiles` y todos los `if (!files.some(x => x.path === f.path))` garantizan que dos inyectores que quieran escribir el mismo `path` no choquen: gana el primero. Igual en `resolveDeps (if (!has(f.path)) files.push(f))`.
- **Idempotencia de SQL.** `addSql(marker, label, sql)` solo appendea si el `marker` (un regex como `/CREATE TABLE IF NOT EXISTS jobs\b/`) no aparece ya en `sql/app.sql`. Dos módulos que declaran la misma tabla no la duplican. Todas las DDL usan `CREATE TABLE IF NOT EXISTS`.
- **Override controlado, no colisión.** Cuando dos caminos *deben* converger, el código resuelve el orden explícitamente. Ejemplo: el vertical `rentals` **sobrescribe** la ruta de booking que el swarm escribió (que la reinventa mal): `deterministicRentalRoutes` reemplaza el archivo por índice (`if (i>=0) files[i] = rf`). Lo mismo `deterministicSwipeRoute` y `deterministicMatchesRoute`, que pisan la versión del LLM con la versión determinista correcta.
- **Normalización de esquema** (`normalizeTables`): detecta y arregla conflictos que el LLM genera — columnas duplicadas en una DDL (SQL inválido: "column specified more than once") y la tabla de match mal modelada (el LLM colapsa `user_a/user_b/item_a/item_b` en un solo FK). La reescribe a la forma canónica para que las rutas deterministas de swipe/match puedan activarse.

El escaneo final (`runSwarmChecks` + `repairPhantomTables`) detecta inconsistencias residuales — tablas-fantasma (una ruta consulta una tabla no declarada), SQL-injection, secretos hardcoded — y las repara con presupuesto acotado. Es la red de seguridad detrás de la prevención por construcción.

8. Overbuild Prevention (YAGNI)

El riesgo simétrico a "faltan features" es "sobran features": un planner ansioso que inyecta medio catálogo. Puglit lo contiene en varias capas:

- **El planner LLM tiene mandato anti-sobre-selección.** El system prompt de `planCapabilities` dice literal "Be precise, do not over-select" y "return ONLY the module names whose capability the product genuinely needs". Corre a `temperature: 0.1` (casi determinista) para no alucinar capabilities por creatividad.
- **Los inyectores deterministas son null -por-defecto.** Cada `deterministicX` devuelve `null` si su regex no matchea. El default es **no inyectar**. Un módulo entra solo si hay evidencia textual (keyword) o evidencia planeada (capability en el summary). No hay inyección "por las dudas".
- **El architect tiene una regla de tamaño con tope contra el over-engineering del modelo.** El prompt de `planBlueprint` dice "SIZE TO THE PRODUCT, do not cap artificially" pero también "never invent unrelated entities like sports leagues in a status page" y, con un lens activo, "stay 100% on THIS product's domain". Una herramienta simple legítimamente puede ser 3-5 tablas; no se la infla.
- **Promociones gobernadas para módulos custom.** Los módulos que el swarm cosechó de builds previos (`harvestModules`) entran como `experimental` y **NO** se auto-inyectan: `findCustomModulesFor` filtra `if (!["stable","core"].includes(m.status)) return false`. Esto evita el *catalog rot* — módulos crudos contaminando builds reales antes de probarse.

El principio operativo es **YAGNI verificable**: si no hay señal (keyword, capability planeada, o dependencia del grafo) que justifique un módulo, no entra.

9. Underbuild Prevention (reference depth, `studyReference`)

El fracaso más común de un generador no es sobre-construir, es **shipear un juguete**: un clon de Promiedos con 4 tablas planas en vez de las 8-15 que el producto real necesita. Puglit ataca esto con dos mecanismos.

9.1 `studyReference` — el Reference Studier

`studyReference(config)` (`web/lib/app-builder.ts`) corre **antes** del blueprint. Es un *Reference Product Analyst* (LLM `premium`, `temperature: 0.2`) que, dado el nombre + pitch, reconstruye **desde memoria** el *fidelity bar* del producto o categoría: las **PÁGINAS distintas**, las **ENTIDADES de datos** detrás de ellas, y las **features-firma** que un usuario notaría faltantes. El prompt es enfático: "list EVERY distinct entity/table the real product needs (think 6-15 for a real product, not 1-2)".

Trae ejemplos literales que calibran la profundidad:

- **Status page** (`status.claude.com / Statuspage`): entidades = `service, component, component_group, status_check, incident, incident_update, maintenance, subscriber`. Páginas = overview (componentes agrupados + barra de uptime de 90 días), historia

de incidentes, incidente individual, mantenimiento programado, suscribirse.

- **Promiedos:** entidades = `competition, team, match, match_event, lineup, player, standing, top_scorer`. Páginas = home (partidos por liga), partido (timeline/stats/lineups), liga (fixture/standings/scorers), equipo (plantel).

Devuelve un bloque de texto (`Reference product` , `REQUIRED DATA ENTITIES` , `SURFACES`) que se pasa como parámetro `reference` a `planBlueprint` . Devuelve `""` solo si la idea es genuinamente novel sin análogo.

9.2 El mandato de profundidad en `planBlueprint`

Cuando hay `reference` , el blueprint lo recibe con una instrucción dura en el user-prompt: *"REFERENCE PRODUCT — the user is cloning this; you MUST reach this depth (model the entities + create the surfaces/pages listed; a blueprint that omits these is a failure)"*. Y en el system-prompt, la sección **REFERENCE-PRODUCT DEPTH:** *"do NOT ship a toy... A live-scores product is NOT 4 flat tables: it needs competitions, matches WITH minute-by-minute events, lineups/ formations, match statistics, team & player pages, multiple standings views, fixtures by round, top scorers."*

A esto se suma la sección **COMPLETENESS (CRITICAL):** por cada tipo de contenido user-generated, incluir *tanto* una ruta CREATE (POST) como una página/form para crearlo — *"never ship a read-only app"*; cada feed se alimenta de una ruta CREATE propia; el chat necesita POST y GET scopeados; el swipe debe detectar el match mutuo atómicamente.

El resultado: el blueprint se planea **a la profundidad del producto real de referencia**, no a la noción vaga del modelo.

10. Case Studies

Cinco familias de producto, mostrando cómo el planner decide en cada una. Las inferencias y keywords están verificadas contra el código.

10.1 Marketplace — "un Tinder para vender cosas usadas" / "un Airbnb para coworkings"

- **Intent / Domain:** `kind` típicamente `accounts` (data por-usuario: mis items, mis bookings).
- **Reference depth** (`studyReference`): para el Airbnb-like, enumera listings, fotos, amenities, availability, bookings, payments, reviews, messages.
- **Blueprint inference:** el ejemplo *Tinder used-goods* del prompt produce `items/swipes/matches/ messages` con detección de match mutuo; `normalizeTables` reescribe la tabla `matches` a la forma canónica `{user_a,user_b,item_a,item_b}` y activa `deterministicSwipeRoute` / `deterministicMatchesRoute` .
- **Module selection:** el ejemplo *rental* dispara el **vertical rentals** (anti-double-booking vía `EXCLUDE USING gist` , pricing en centavos, refund-by-policy-snapshot, reviews double-blind) y **sobrescribe** la ruta de booking del swarm con la verificada. Keywords `reserv|rental|alquiler| disponibilidad` también pueden activar `booking` / `reviews` .
- **Dependency expansion:** si hay `payments` , `resolveDeps` fuerza `crypto` .

10.2 CRM — "un HubSpot simple para mi pyme"

- **Intent / Domain:** `accounts` , multi-usuario; muy probablemente B2B.
- **Module selection:** `crm` matchea por `bp.summary` / tablas (`leads/contacts/deals/pipeline`). `planCapabilities` típicamente agrega `multitenancy` (orgs/teams — *"an ERP needs ... multitenancy ... even if unsaid"* aplica igual a un CRM B2B), `forms` (intake/lead capture), `marketing` / `helpdesk` según el pitch.
- **Dependency expansion:** `forms` → `validation` . Si el CRM emite notificaciones a sistemas externos, `webhooksout` → `crypto` , `queue` .

10.3 ERP — "un ERP para hospitales"

Este es el **caso testigo** que motiva el capability planner: el texto *no contiene* keywords de `auditlog` , `multitenancy` ni `entitlements` .

- **Domain classification:** `accounts` .
- **Capability extraction:** aquí brilla `planCapabilities` . El prompt instruye explícitamente *"an ERP needs auth+multitenancy+audit even if unsaid"*. El planner devuelve, p.ej., `["multitenancy","auditlog","migrations","admin","entitlements"]` , que se appendean a `bp.summary` como `[capabilities: multitenancy auditlog migrations admin entitlements]` .
- **Module selection:** ahora los `deterministicX` de esos módulos **sí** matchean, porque su haystack incluye `bp.summary` . Sin el planner, ninguno hubiera disparado — esta es la crítica *"keyword injection is fragile"* resuelta en la práctica.

10.4 AI Agent — "un asistente tipo JARVIS"

- **Domain:** `accounts` (memoria + identidad por-usuario).

- **Module selection:** `deterministicAgent` matchea `asistente|assistant|jarvis|copilot|ai agent| chatbot|\bbot\b|second brain` sobre `name+tagline+summary`. Inyecta `lib/agent/brain.ts` (el patrón NanoClaw: identidad cross-channel, memoria persistente, tool-calling).
- **Dependency expansion:** `MODULE_REQUIRES.agent = ["llm","crypto"]`. `resolveDeps` detecta `lib/agent/brain.ts` (vía `PRIMARY["agent"]`), calcula el cierre y **force-inyecta** `llm` y `crypto` — el agente nunca shipea sin su cliente LLM ni sin cifrado para sus tokens MCP.
- **Capabilities complementarias:** si el pitch menciona docs/knowledge, `planCapabilities` suele agregar `rag` (→ `llm`) y/o `graphify` / `memorygraph`.

10.5 Fintech — "una wallet de créditos para una app"

- **Domain:** `accounts`.
- **Module selection:** `wallet` matchea `credit|wallet|billetera|saldo|balance|loyalty|prepaid| recarga` y aporta el **ledger append-only** (`credit/debit` atómicos, balance = sum, sin negativos). El haystack de `wallet` incluye `config.monetization`, así que un modelo de negocio de "créditos/puntos" lo dispara aunque el nombre no lo diga. En paralelo, `twofa` matchea `fintech|banking|wallet|secure login`, y `auditlog` matchea `fintech|banking|compliance|soc2|gdpr` — el planner suele reforzar los tres por la naturaleza regulada del dominio.
- **Dependency expansion:** si hay `payments / billing`, `resolveDeps` fuerza `crypto` para cifrar los secretos de proveedor; `twofa` usa el mismo `crypto` para cifrar el TOTP secret.
- **Underbuild prevention:** una app fintech *no* es read-only; la sección COMPLETENESS exige rutas CREATE (recargar, debitar) con sus páginas, y las mutaciones multi-statement (debitar + registrar) van en una transacción (`BEGIN/COMMIT/ROLLBACK`) por el estándar de backend del prompt.

Cierre

El Capability Planner de Puglit es la unión de tres decisiones encadenadas — **clasificación de dominio** (`kind`), **extracción de capabilities** (regex determinista + LLM planner que alimenta el mismo punto de inyección), y **expansión por grafo** (`MODULE_REQUIRES` + cierre transitivo) — con dos guardrails simétricos: **reference depth** (`studyReference`) contra el underbuild y **YAGNI verifiable** contra el overbuild. Lo que decide *qué construir* nunca es un solo LLM adivinando: es un planner híbrido donde cada inferencia tiene una señal verificable detrás (un keyword, una capability planeada, o una arista del grafo de dependencias), y donde lo determinista y lo probabilístico convergen en `finalize`.